

Indie Cloud System Report

Final Project Documentation

Prepared by:

Milan Ramsawmy

Shivali Sharma

Pratik Byanthal

Kevin Liu

Course: cs4471A/cs9549A - Software Architectures

Instructor: Nazim Madhavji

University: Western university

December 6, 2025

Contents

1	Introduction	2
2	Requirements	2
2.1	System-Wide Functional Requirements	2
2.2	Microservice-Specific Requirements	2
2.2.1	Microservice 1: Streaming Videos	2
2.2.2	Microservice 2: Review Section	2
2.2.3	Microservice 3: Linktree for Creators	3
2.2.4	Microservice 4: Leaderboard	3
2.2.5	Microservice 5: Merch Advertisement	3
2.3	Architecturally Significant Requirements (ASRs)	3
3	Architecture and Design	4
3.1	Context Model	4
3.2	Overall System Architecture	5
4	Implementation Details	6
5	Testing	7
5.1	Testing Methodology	7
5.2	Key Test Cases	7
6	Operational Evidence	7
6.1	Service Discovery Flow	8
6.2	Microservice Interaction	8
7	Operational Evidence	8
7.1	Microservice User Flow	8
8	Appendix	10
9	References	10

1 Introduction

This report documents the final state of the **Indie Cloud** system, a microservices-based platform designed to facilitate the distribution and discovery of cloud-based games. The system mimics a digital storefront, allowing users to authenticate, search a game catalog, and interact with specific game services including streaming, reviews, and leaderboards.

The project adopts a cloud-native approach, utilizing a Service Registry for dynamic discovery and a suite of decoupled microservices to ensure scalability, modifiability, and fault tolerance.

2 Requirements

2.1 System-Wide Functional Requirements

The core functional requirements (FRs) of the Indie Cloud infrastructure are:

- **FR1 - User Management:** Users can create accounts, log in/out, and reset passwords.
- **FR2 - Developer Onboarding:** Developers can register as creators and upload game builds, screenshots, and descriptions.
- **FR3 - Catalog Browsing:** Players can browse or search for games by tags, genre, price, or rating.
- **FR4 - Game Download:** Users can securely download games.
- **FR5 - Game Updates:** Developers can upload new versions of their games.

2.2 Microservice-Specific Requirements

The system is composed of five distinct microservices, each with specific requirements:

2.2.1 Microservice 1: Streaming Videos

- **FR1:** When a user opens a game page, the service fetches active streams via the Twitch API (or mock) and selects the top streamer.
- **FR2:** Displays the stream via an embeddable link or iframe. If no streamer is live, it returns "No active streams".

2.2.2 Microservice 2: Review Section

- **FR1:** Game authors can enable or disable the review section for their game.
- **FR2:** Registered users can post text reviews and ratings (1-5 stars).

2.2.3 Microservice 3: Linktree for Creators

- **FR1:** Developers can create a profile with up to 5 resource links (label and URL) per game.
- **FR2:** Links are displayed on the game page and must open in a new browser tab.

2.2.4 Microservice 4: Leaderboard

- **FR1:** Accepts POST requests with game ID, user ID, and score; validates existence via auth/game services.
- **FR2:** Provides a GET endpoint to fetch the top N (5-10) players sorted by rank.
- **FR3:** Updates scores only if the new submission is higher than the previous best (one score per user).

2.2.5 Microservice 5: Merch Advertisement

- **FR1:** Developers can create merch ads with start and end dates; ads automatically expire.
- **FR2:** Ads can link to external storefronts (e.g., Etsy, Shopify).

2.3 Architecturally Significant Requirements (ASRs)

The following quality attributes drove the architectural decisions:

Attribute	Requirement Metric	Architectural Justification
Scalability (Streaming)	Must handle $\geq 50,000$ concurrent users for game lookups.	Requires efficient caching and potentially asynchronous microservices to handle high throughput.
Scalability (Reviews)	Must handle 100,000 reviews per game and 10,000 concurrent reads.	Justifies the use of NoSQL/distributed databases (e.g., MongoDB) and read-caching.
Availability	99.9% uptime per month with Max Time To Recovery (MTTR) ≤ 10 minutes.	Requires redundancy (backup registry) and graceful degradation if external APIs (like Twitch) fail.
Performance	General browsing must handle 1,000 users with response time ≤ 2 s.	Determines database optimization strategies and use of CDNs.
Modifiability	Changes to Linktree logic should not require modifying other services.	Enforces loose coupling between the Linktree service and the main game page.

Table 1: Architecturally Significant Requirements (ASRs)

3 Architecture and Design

3.1 Context Model

The Context Model (Figure 1) depicts the Indie Cloud system boundary. It treats the system as a black box to illustrate interactions with external entities such as Game Creators, Players, and the Twitch API.

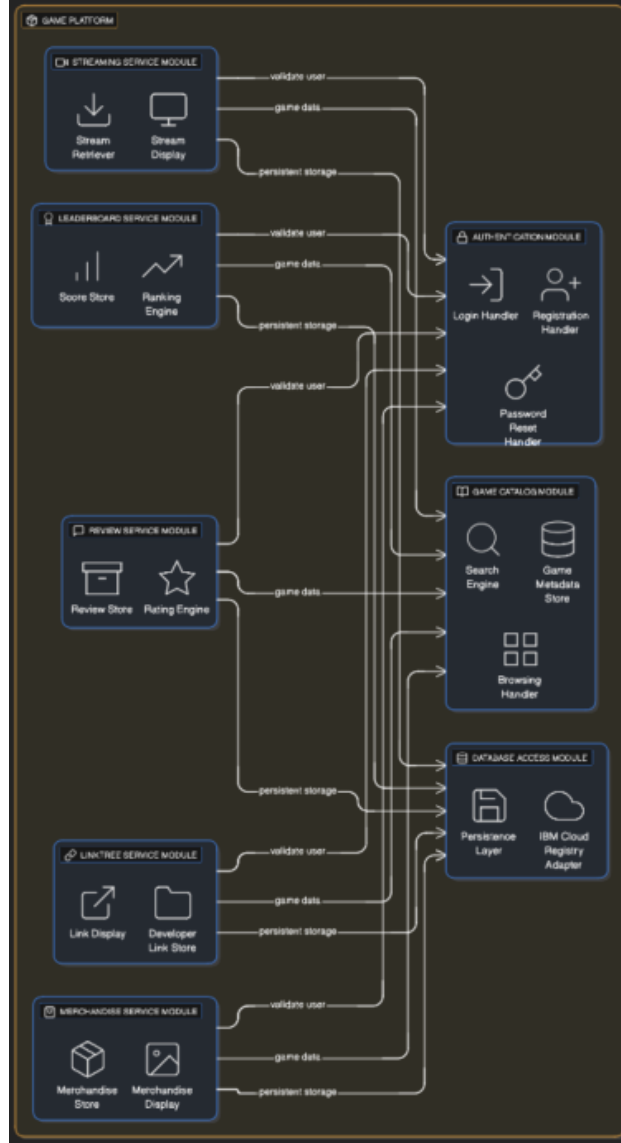


Figure 1: Context Model: External Interactions

3.2 Overall System Architecture

Figure 2 illustrates the internal decomposition of the system. The architecture is divided into three main components:

1. **Service Consumer (Front-end):** The GUI built with react native where users search for games. It queries the Registry to locate the active microservices.
2. **Service Registry:** The discovery server that maintains the locations (IP/Port) of all active microservices.
3. **Services Layer:** The independent microservices that execute business logic:
 - **Streaming Service:** Fetches live video data.

- **Review Service:** Manages user ratings and comments.
- **Linktree Service:** managing creator resource links.
- **Leaderboard Service:** Manages high scores and user rankings.
- **Merch Service:** Handles advertisement expiration and redirection.

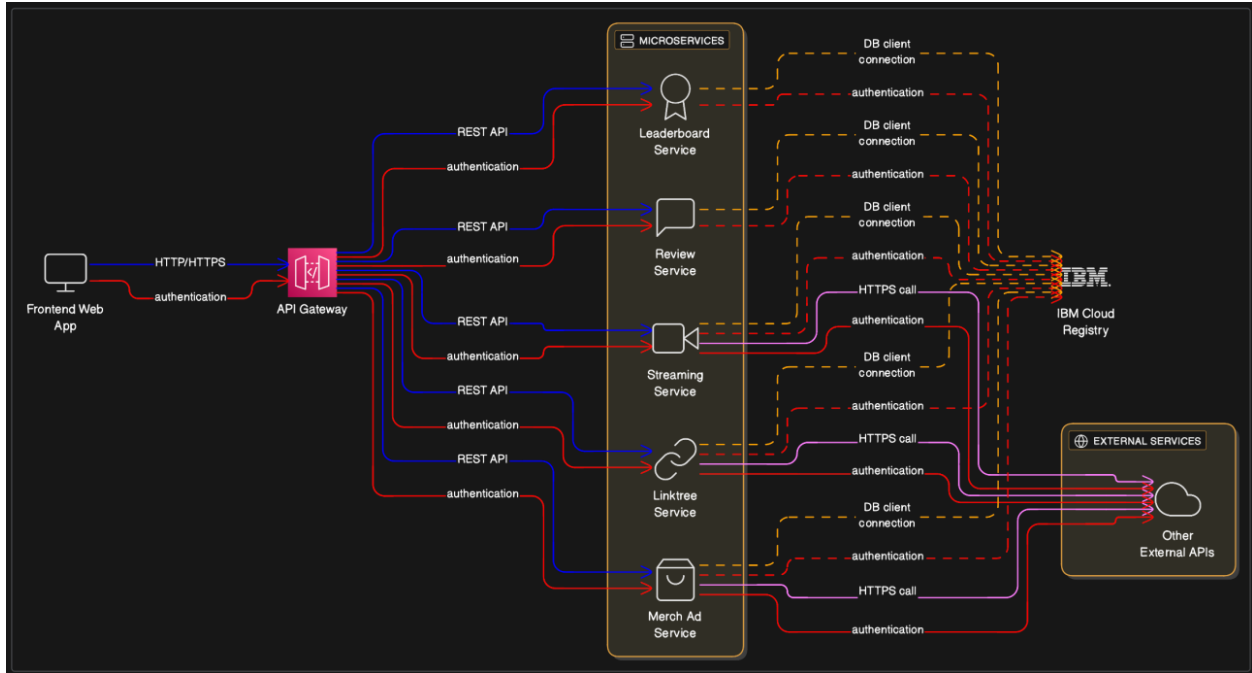


Figure 2: System Architecture: All requests are routed through the Service Registry to the specific Microservices (Streaming, Reviews, Linktree, Leaderboard, Merch).

4 Implementation Details

The system was implemented using the following tools and technologies:

- **Language:** Node.js (Express).
- **Cloud Platform:** IBM Cloud (Code Engine).
- **Database:** IBM cloudant for ease of deployment and scalability
- **Containerization:** Docker.
- **Version Control:** GitHub.

5 Testing

5.1 Testing Methodology

- **Unit Tests:** Verified input sanitization for the Linktree service to ensure 0 unescaped HTML tags.
- **Performance Testing:** Verified that Leaderboard updates reflect within 3 seconds.
- **Resilience Testing:** Simulating a crash of the Primary Registry to verify failover.

5.2 Key Test Cases

Test ID	Description	Result
TC-01	Streaming: Query game with no live streams. Expect "No active streams" message.	Pass
[cite _{start}]TC – 02	Merch: Verify ad automatically expires after end date[cite: 154].	Pass
TC-03	Linktree: Attempt to add 6th link. Expect error (Limit 5).	Pass
TC-04	Leaderboard: Submit lower score than current best. Expect no database update.	Pass

Table 2: Summary of Key Test Results

6 Operational Evidence

This section demonstrates the user interaction flow. Please see next page.

6.1 Service Discovery Flow

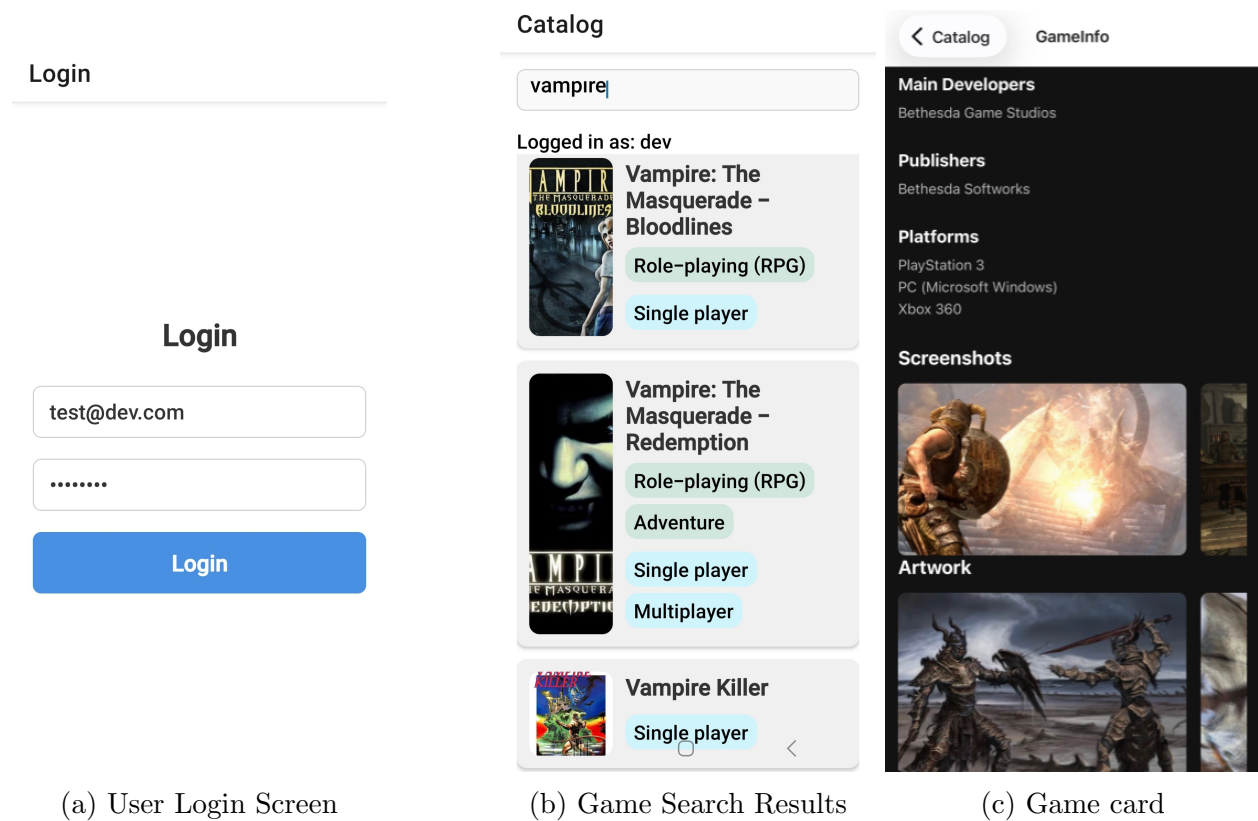


Figure 3: Mobile Interface: Operational evidence showing the authentication and discovery flow.

6.2 Microservice Interaction

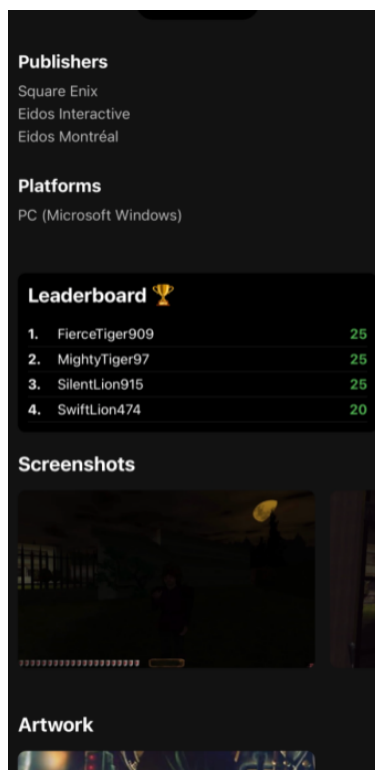
7 Operational Evidence

7.1 Microservice User Flow

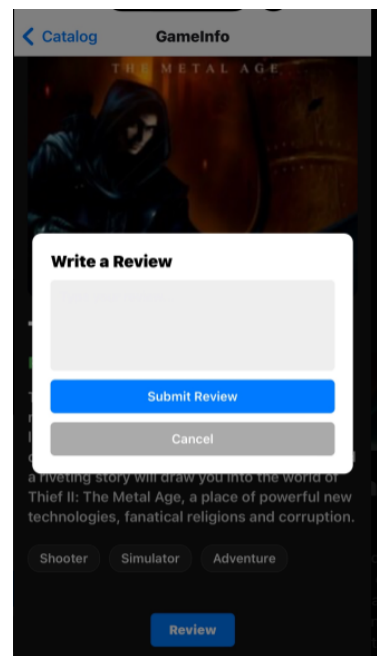
The following screenshots demonstrate the complete user journey through the Indie Cloud system, from discovery to interaction with specific microservices.



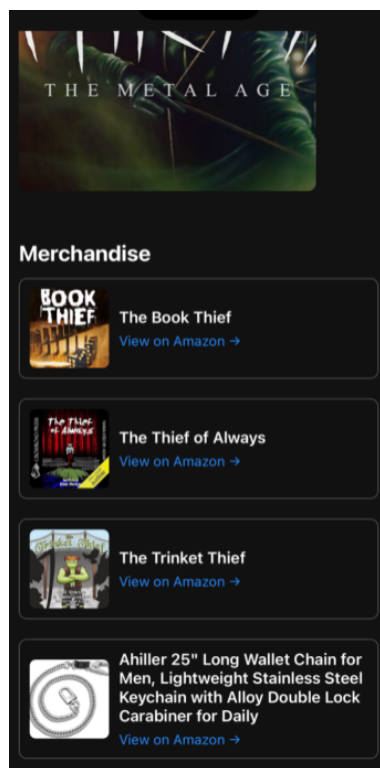
(a) Streaming MicroService



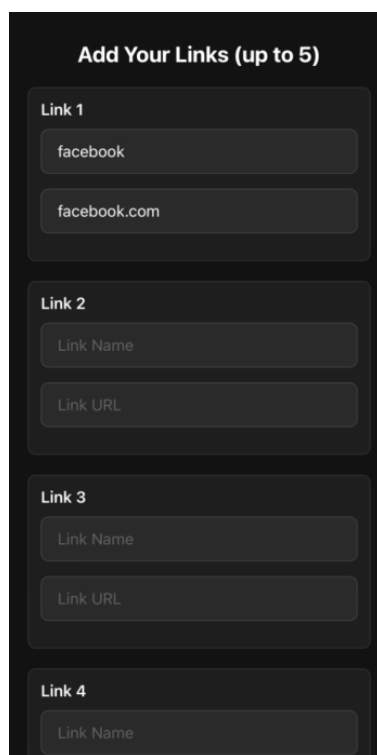
(b) LeaderBoard Microservice



(c) Review Microservice



(a) Merchandise MicroService



(b) Linktree MicroService

8 Appendix

- GitHub Repository: <https://github.com/thecoderpb/IndieCloud.git>

9 References

1. IBM Cloud Documentation. Available: <https://cloud.ibm.com/docs>
2. Docker Documentation. Available: <https://docs.docker.com>
3. Project Requirements Document (PR2).